

System Design Document

Scalable Discovery of Satellite Imaging Opportunities

for a Low Earth Orbit (LEO) Imaging Satellite Constellation

Prepared as a response to the System Design Exercise (Exercise 04)

Version 1.1

September 2026

1. Executive Summary

The company operates a growing constellation of camera-mounted imaging satellites tasked with photographing other satellites, rocket bodies, and debris (“target objects”) in low Earth orbit. This document proposes a cloud-native, serverless-first architecture on AWS that computes, every six hours, the complete set of imaging opportunities between every imaging satellite and every target object over a rolling seven-day horizon, and serves that data back to consumers with sub-second query latency.

The core engineering challenge is combinatorial scale, not algorithmic difficulty — the opportunity-finding logic itself is a provided black box. At today’s scale (50 satellites × 30,000 objects) a naive sweep already requires 1.5 million black-box invocations per run; by the stated 2026 targets (250 satellites × 60,000 objects) that grows to 15 million, repeated four times a day. Sequential execution is impossible at every point on the roadmap (8.68 days for one end-of-2026 pass at an assumed 50 ms per call), so the design combines three levers: (1) massive parallelism — AWS Step Functions Distributed Map fanning batches out to Lambda workers — which is what meets the six-hour deadline and needs only about 16 concurrent workers at the baseline assumptions; (2) a cheap orbital pre-filter that prunes geometrically implausible pairs, cutting compute cost by roughly 6.7× and providing margin if the black box is slower than assumed; and (3) a DynamoDB-backed serving layer keyed for the two access patterns the business needs, with single-digit-millisecond reads and an atomic, validated cutover between runs.

Because the output drives tasking decisions and the pipeline runs unattended four times a day, correctness is protected by automated testing at every level (Section 14) — including a quality gate inside each production run that must pass before new results go live. The result is a solution that meets the 6-hour cadence with wide margin, scales close to linearly as the constellation and catalog grow through 2026 and beyond, and keeps ongoing cost proportional to actual usage (about \$1,000 per month at end-of-2026 scale under the stated assumptions).

Deliverables at a Glance

This document explicitly addresses the five areas requested for this exercise. For quick reference:

Required item	Where to find it in this document
Diagrams	Six figures: sensitivity chart (Fig. 1, §5.7), components and data flow (Fig. 2, §6), data model (Fig. 3, §7), run sequence (Fig. 4, §8), test pyramid (Fig. 5, §14.1) and CI/CD pipeline (Fig. 6, §14.3)
Assumptions	Section 4 — ten numbered assumptions (A1–A10), each with the impact if it proves wrong and how it will be validated
Calculations & Estimations	Section 5 — pair counts, sequential baseline, pre-filter impact, parallel sizing, storage and query sizing, cost estimate, and a sensitivity analysis of the two least certain inputs
Trade-offs	Section 13 — seven decisions compared side by side, with what each choice gives up
Automated Testing	Section 14 — test layers, runtime quality gate, CI/CD gates and acceptance criteria, example tests

2. Problem Statement

Design a system that, for a constellation of imaging satellites in LEO, identifies every imaging opportunity — a moment where an imaging satellite and a target object pass within 100 km of each other — across a seven-day look-ahead horizon. An existing R&D function, treated as a black box, already computes this for a single (imaging satellite, target object) pair:

```
imaging_datetimes: list[datetime] = find_imaging_opportunities(
    imaging_satellite_id: str, target_object_id: str)
```

The system must run this computation for the entire constellation against the entire tracked catalog of target objects, on a fixed schedule, and make the resulting opportunities available for very fast lookup.

3. Requirements

3.1 Functional Requirements

- Compute all imaging opportunities for every (imaging satellite, target object) pair over a 7-day horizon.
- Treat `find_imaging_opportunities()` as an opaque, unmodifiable black box.
- Re-run the full computation every 6 hours so orbital drift is reflected promptly.
- Persist results so they can be fetched and queried by consumers (e.g. mission planning, tasking systems).

3.2 Non-Functional Requirements

- Hosted on a public cloud provider, preferably AWS.
- Query latency for stored results must be sub-second.
- Scalable from $50 \times 30,000$ today to at least $250 \times 60,000$ by end of 2026 without a redesign.
- Cost should scale with workload rather than requiring permanently reserved capacity.
- The 6-hour computation window must complete reliably with headroom to spare for retries.

4. Assumptions

The design and the sizing calculations in Section 5 rest on the following explicit assumptions. Where a number is assumed rather than known, it is a planning value to be replaced by a measurement; the third column says what happens if the assumption is wrong and how it will be checked. The two most influential are A1 and A2, and Section 5.7 shows the design remains viable across a wide range of both.

ID	Assumption	If wrong / how validated
A1	Black-box latency: <code>find_imaging_opportunities()</code> takes about 50 ms per call (plausible for SGP4-class propagation over a 7-day window). Planning value; 250 ms and 1 s are analysed as sensitivities.	Drives worker concurrency and compute cost linearly. Validate first: time the function on a few thousand real pairs. See §5.7.
A2	Pre-filter reduction: coarse orbital screening removes about 85% of raw pairs. This is a planning value that has not yet been measured on the real catalog.	Affects cost (not the deadline). Measured by the recall/reduction test in §14.1. The design still works at 0% (§5.7).

ID	Assumption	If wrong / how validated
A3	Opportunities per candidate pair: about 1.5 on average (including pairs with none), assuming the function returns one datetime per distinct opportunity. Independent of whether the pre-filter is used.	Drives DynamoDB writes and storage linearly. If the function returns time-sampled datetimes, writes could be ~10× higher (\$5.8).
A4	Fresh orbital elements (TLE-equivalent) for all imaging satellites and tracked objects are available at least every 6 hours, from an internal catalog or a provider such as Space-Track.	A stale snapshot means stale results; the run records the snapshot age and alarms if it exceeds a threshold.
A5	Per the exercise, no race conditions or concurrency conflicts need to be handled for reads or writes. The generation pointer (§7.3) and TTL are included as low-cost good practice, not as conflict resolution.	None for the exercise scope. If concurrent writers were introduced, conditional writes would be added.
A6	The black box is stateless, side-effect-free, deterministic for identical inputs and orbital snapshot, and safe to call concurrently and repeatedly.	Idempotent retries and golden-file tests depend on determinism; the contract tests in §14.1 check it.
A7	AWS is the target platform, per the exercise's stated preference.	Design uses standard managed services; portability effort would be moderate.
A8	The satellite and target-object catalog, and permission to query orbital data sources, are available to the team.	Blocking dependency; confirm at project start.
A9	Worker configuration: Lambda x86 with 2 GB memory, 75 pairs per batch, about 0.25 s of I/O overhead per batch; Distributed Map child workflows use the Express type (5-minute limit per batch).	Batch size must shrink if per-call latency exceeds ~12 s (Lambda 15-minute limit) or ~4 s (Express 5-minute limit).
A10	Storage and pricing basis: each item is under 1 KB (about 350 B stored), the GSI projects all attributes (2 write units per item), and prices are us-east-1 list prices checked in September 2026 (Lambda \$0.0000166667 per GB-s + \$0.20 per million requests; DynamoDB on-demand \$0.625 per million write units; Step Functions Express \$1.00 per million requests).	Costs are estimates, not quotes. Some third-party sources still quote the pre-November-2024 DynamoDB price (\$1.25); at that price the write line doubles. Re-check with the AWS pricing calculator.

5. Calculations & Estimations

Sizing the problem correctly is the most important design input. This section derives the workload from the assumptions in Section 4, shows the arithmetic, and tests how sensitive the conclusions are to the two least certain assumptions (A1 and A2). All costs are estimates at us-east-1 list prices.

5.1 Pair-Count Scale

Raw pairs = imaging satellites × target objects.

Milestone	Imaging satellites	Target objects	Raw pairs per run
Today	50	30,000	1,500,000
End of 2025	250	30,000	7,500,000
End of 2026	250	60,000	15,000,000

5.2 Sequential Baseline Runtime

Using assumption A1 (50 ms per call), a sequential sweep of every raw pair would take:

End of 2026:	15,000,000 pairs × 50 ms = 750,000 s ≈ 208.3 h ≈ 8.68 days
	(34.7× the 6-hour budget)
Today:	1,500,000 pairs × 50 ms = 75,000 s ≈ 20.8 h

Even today a sequential sweep does not fit in six hours, and at 250 ms or 1 s per call the end-of-2026 sweep would take 43.4 or 173.6 days. Sequential evaluation is therefore ruled out at every point on the roadmap. The design question is how much parallelism, and how much reduction, is needed — answered in Sections 5.3, 5.4 and 5.7.

5.3 Effect of Pre-Filtering on Candidate Count

Applying assumption A2 (85% reduction), the number of pairs that actually require a black-box call drops sharply:

End of 2026:	15,000,000 × (1 − 0.85) = 2,250,000 candidate pairs
Today:	1,500,000 × (1 − 0.85) = 225,000 candidate pairs

Every pair removed is a black-box call, and a share of a Lambda invocation, that is no longer paid for. Because a correct filter removes only pairs that would have produced no opportunities, it does not change what is stored: the number of DynamoDB items is the same with or without it (Section 5.5).

5.4 Parallel Compute Sizing

With 2,250,000 candidate pairs (end-of-2026 scale) and a batch size of 75 pairs per Lambda invocation (A9):

Batches needed:	2,250,000 ÷ 75 = 30,000 batches
Black-box time:	75 × 50 ms = 3.75 s per batch (about 4.0 s billed)
Total worker time:	30,000 × 3.75 s = 112,500 s (≈ 31.25 hours)

Targeting a conservative 2-hour compute window (leaving 4 of the 6 hours as headroom for data fetch, filtering, retries, validation and orchestration overhead):

Required concurrency =	112,500 s ÷ 7,200 s ≈ 15.6 → ~16 concurrent workers
------------------------	---

For context, Lambda's default concurrency quota is 1,000 executions per Region (raisable on request) and a Distributed Map can run up to 10,000 child executions in parallel, so 16 is far below any limit. At 1,000 concurrent workers the compute stage would take about two minutes (30,000 batches ÷ 1,000 × 4 s); the 2-hour figure is a planning ceiling, not the expected runtime. In practice the end-to-end run is dominated by ramp-up, snapshot and pre-filter time and the quality gate, so we plan for tens of minutes and alarm if a run exceeds three hours.

5.5 Storage, Write Throughput and Query Sizing

Using assumptions A3 and A10:

Items written per run:	2,250,000 × 1.5	= 3,375,000 (≈ 13.5 million per day)
Write units per run:	3,375,000 × 2 (table+GSI)	= 6,750,000 WRU
Stored per generation:	3,375,000 × ~350 B	≈ 1.2 GB table + 1.2 GB GSI
Retained (24 h TTL):	4 generations	≈ 9.4 GB in total
Items per satellite:	3,375,000 ÷ 250	= 13,500 per generation (≈ 4.7 MB)
Items per object:	3,375,000 ÷ 60,000	≈ 56 per generation

- Query sizing. A 24-hour window for one satellite is about 13,500 ÷ 7 ≈ 1,930 items (≈ 0.68 MB), which fits in one Query page (1 MB limit) and returns in tens of milliseconds. A full 7-day window is about five pages. The API therefore requires a time window (default: next 24 hours) and paginates longer ranges,

which is how the sub-second requirement is kept for every request. A 24-hour query consumes about 85 read units (eventually consistent), roughly \$0.00001.

- Hot-partition control. One partition key can sustain about 1,000 write units per second, so writing one satellite's 13,500 items takes at least ~14 s. If the manifest were ordered by satellite, hundreds of workers would hit the same few partitions and be throttled. The manifest is therefore shuffled so every batch mixes satellites; a 10-minute ingest then averages about 5,600 items per second spread over 250 partitions (~22 per partition per second).
- Deletion is free. Expired generations are removed by DynamoDB TTL, which does not consume write capacity.

5.6 Cost Estimate

The figures below are planning-stage estimates for end-of-2026 scale under assumptions A1–A3, A9 and A10, with 4 runs per day (120 per month):

Component	Basis	Per run	Per month
Lambda workers	30,000 batches × 4.0 s × 2 GB = 240,000 GB-s at \$0.0000166667, plus 30,000 requests at \$0.20 per million	≈ \$4.01	≈ \$481
DynamoDB writes	6.75 million write units at \$0.625 per million (on-demand, us-east-1)	≈ \$4.22	≈ \$506
DynamoDB storage	~9.4 GB retained at \$0.25 per GB-month	—	≈ \$2
Step Functions (Express children)	30,000 child executions at \$1.00 per million, plus about \$0.13 of duration charges	≈ \$0.16	≈ \$19
S3, Glue/Athena, API Gateway, CloudWatch	Workload-dependent; small relative to the above; not modelled	—	minor
Total (modelled)		≈ \$8.4	≈ \$1,000

- Using Standard instead of Express child workflows would add about \$3 per run in state transitions (≈ \$360 per month at roughly four transitions per child), which is why Express is specified.
- If the older \$1.25 per million DynamoDB write price were still in effect, the write line would double to about \$1,010 per month.
- Without the pre-filter the Lambda line rises to about \$3,205 per month (200,000 batches per run); the DynamoDB line is unchanged because the same opportunities are stored.

5.7 Sensitivity Analysis

Assumptions A1 (per-call latency) and A2 (pre-filter reduction) are the least certain and the most influential. Figure 1 and the table below show, for end-of-2026 scale, the concurrency needed to finish the compute stage in a 2-hour window and the resulting monthly Lambda cost, across three latencies and five levels of pre-filter effectiveness (black-box time only in the chart; I/O overhead is included in the cost figures).

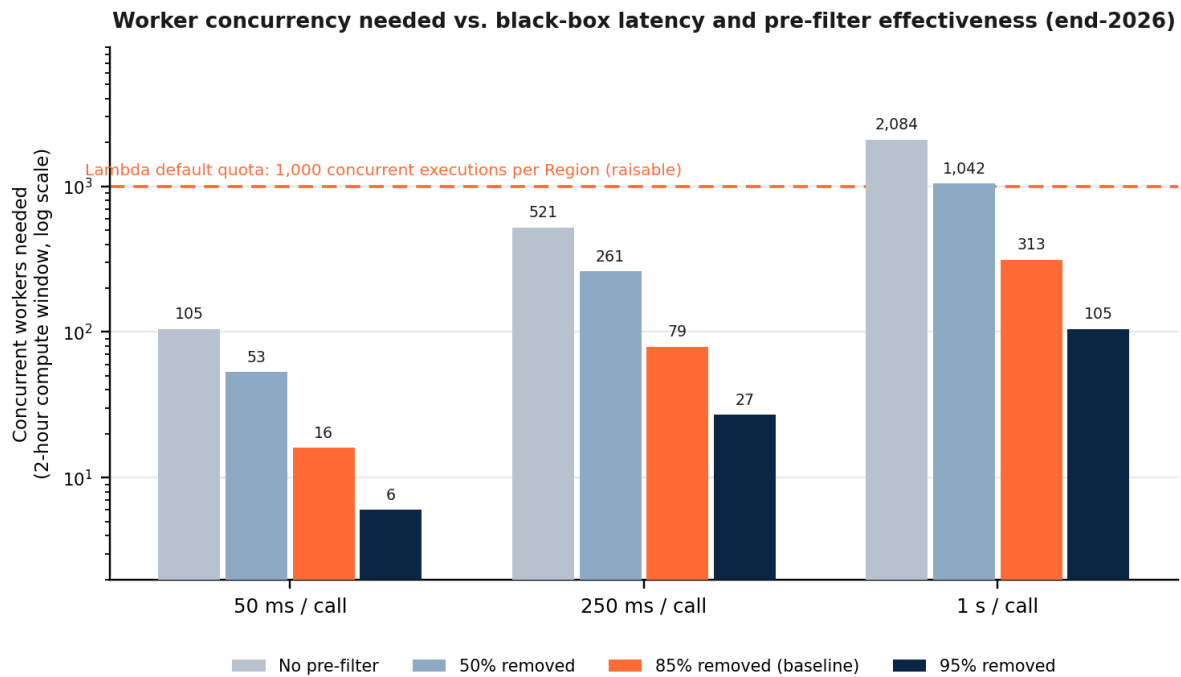


Figure 1. Worker concurrency needed (2-hour window) versus black-box latency and pre-filter effectiveness.

Pre-filter removes	50 ms per call	250 ms per call	1 s per call
None (parallel brute force)	105 workers · \$3,205/mo	521 workers · \$15,205/mo	2,084 workers · \$60,205/mo
50%	53 workers · \$1,602/mo	261 workers · \$7,602/mo	1,042 workers · \$30,102/mo
70%	32 workers · \$961/mo	157 workers · \$4,561/mo	625 workers · \$18,061/mo
85% (planning baseline)	16 workers · \$481/mo	79 workers · \$2,281/mo	313 workers · \$9,031/mo
95%	6 workers · \$160/mo	27 workers · \$760/mo	105 workers · \$3,010/mo

- The six-hour deadline is met in every case examined. The worst case (no filter, 1 s per call) needs about 2,084 concurrent workers — above the default Lambda quota, so a quota increase would be requested, but well inside Distributed Map's 10,000 limit.
- The pre-filter is primarily a cost and risk control: at the baseline it reduces Lambda spend by about 6.7× (from ~\$3,205 to ~\$481 per month), and the benefit grows with slower black boxes (at 1 s per call, ~\$60,205 versus ~\$9,031 per month).
- Even if the pre-filter proves only 50% effective, the design remains comfortable at 50 ms and 250 ms per call.

5.8 What the Numbers Mean

- Parallelism meets the clock; the filter controls the bill. About 16 workers are enough at baseline, and no scenario examined exceeds about 2,100.
- The two measurements that matter most are A1 and A2. Both can be obtained in a day on the real catalog and should be taken before the build is committed.
- Storage cost is independent of the filter and scales linearly with A3. If the black box returns time-sampled datetimes rather than one per opportunity (say ten times A3), write cost would be about

\$5,000 per month; that outcome would trigger a move to one item per pair, or interval compression (Section 13).

6. Proposed Architecture

The architecture is organized into four stages: (1) scheduling and orbital data preparation, (2) candidate pair pre-filtering, (3) massively parallel opportunity computation, and (4) low-latency persistence and serving — with an automated quality gate between computation and publication. All components are managed AWS services, chosen to minimize operational overhead and to scale automatically with load.

Components and data flow

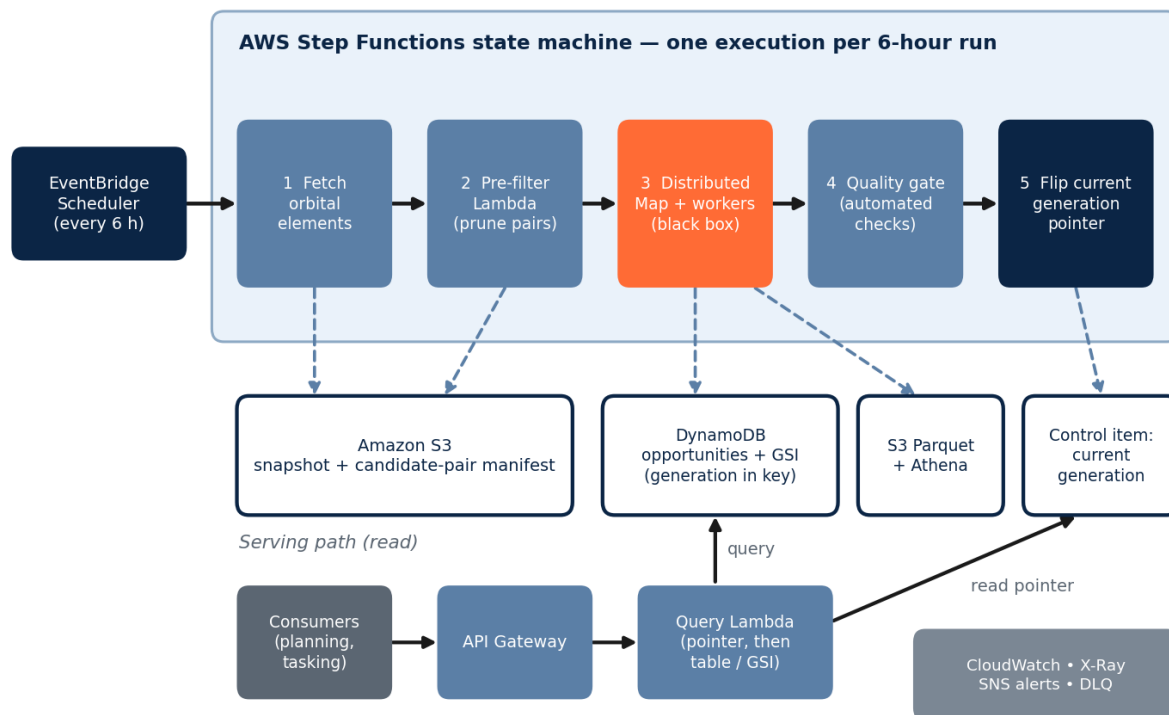


Figure 2. Components and data flow for the imaging opportunity discovery pipeline.

6.1 Scheduling & Orbital Data Preparation

An Amazon EventBridge Scheduler rule fires every 6 hours and starts a Step Functions state machine that orchestrates the entire run end to end. The first state fetches the latest orbital elements for all 250 imaging satellites and up to 60,000 target objects from the company’s tracking catalog (or an external source such as Space-Track) and stores a timestamped snapshot in S3. Anchoring every run to a fresh snapshot is what lets the system reflect orbital drift within a single 6-hour cycle, which is the entire point of re-running the computation periodically. The run’s start time (for example 2026092806) becomes its generation ID.

6.2 Candidate Pair Pre-Filtering

Two objects can only produce an imaging opportunity (a 100 km approach) if their orbits overlap in the relevant altitude band and their orbital planes bring them into proximity at some point in the 7-day window. A lightweight pre-filter Lambda (or Fargate task, if the catalog grows large enough to need more memory) computes coarse orbital envelopes — perigee/apogee altitude bands and inclination groupings — for every satellite and object from the snapshot, and eliminates pairs that cannot come within 100 km. This is a standard

technique borrowed from conjunction screening in space situational awareness; the planning assumption is that it removes about 85% of the naive set (A2), to be measured on the real catalog.

The filter must be conservative rather than clever: it keeps a pair whenever there is doubt, uses a margin larger than 100 km to allow for orbit changes over the horizon, and is validated by an automated recall test (Section 14.1) that fails the build if it ever drops a pair the black box would have flagged. The surviving candidate pairs are written to S3 as a manifest (newline-delimited JSON or CSV, partitioned into chunks) and shuffled so consecutive batches mix satellites. This manifest is the sole input to the expensive stage that follows.

6.3 Massively Parallel Opportunity Computation

A Step Functions Distributed Map state reads the S3 manifest directly and fans out execution across parallel child workflows, with automatic batching, checkpointing, per-batch retry and a configurable failure threshold, without any custom queue-management code. Each child runs a short-lived Lambda function that receives a batch of candidate pairs (75 in the baseline), calls `find_imaging_opportunities()` for each pair, and writes the results to DynamoDB with `BatchWriteItem` (25 items per call).

Batching keeps the invocation count manageable (2.25 million candidate pairs at 75 per batch is 30,000 invocations rather than 2.25 million) while allowing as many batches to run concurrently as needed. Child workflows use the Express type: each batch runs for seconds, well inside the five-minute Express limit, and Express keeps orchestration cost to cents per run. If profiling shows the black box is CPU- or memory-heavy, or slow enough to approach Lambda's 15-minute limit, the same manifest-driven fan-out can be re-pointed at AWS Batch or Fargate with no change to the upstream stages — the pre-filter and manifest format are compute-engine agnostic by design.

6.4 Persistence & Serving Layer

Results are written to Amazon DynamoDB, chosen because the query patterns — “get opportunities for satellite X” and “get opportunities for object Y” — are key-based lookups rather than analytical aggregations, and DynamoDB delivers predictable single-digit-millisecond latency without capacity planning. On-demand capacity mode suits the bursty write pattern (a burst of writes every 6 hours followed by a read-heavy period).

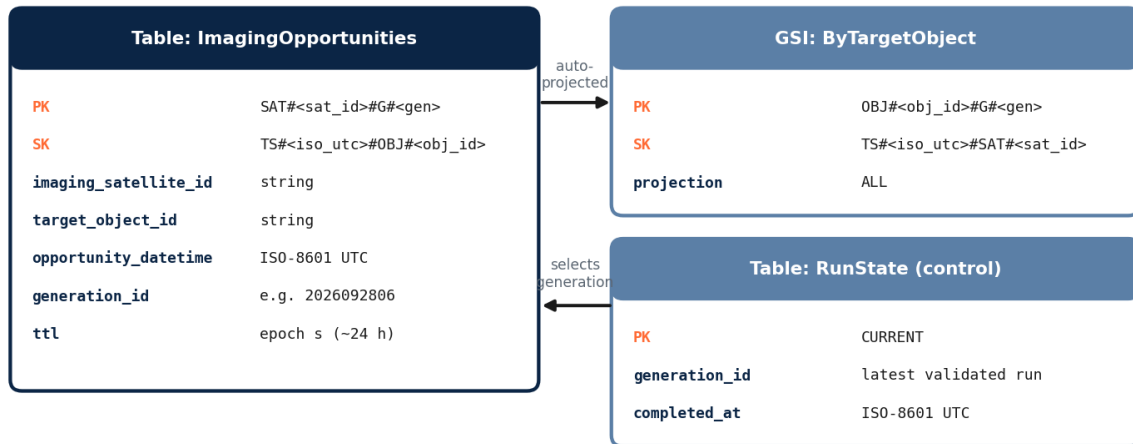
A lightweight API layer (Amazon API Gateway plus a small query Lambda) exposes opportunities for a given imaging satellite, for a given target object, and for a specific pair. The query Lambda first reads the current-generation pointer (cached briefly), then queries the table or index for that generation within a bounded time window (Section 7.4). End-to-end latency is expected in the tens of milliseconds, comfortably inside the sub-second requirement.

In parallel with the DynamoDB write, each run also lands a full copy of its results in S3 in a columnar format (Parquet), partitioned by run date and cataloged with AWS Glue for Amazon Athena. This supports historical analysis, auditing and trend reporting without placing any load on the hot serving path.

7. Data Model

The DynamoDB model is built around the two access patterns that matter operationally — “all opportunities for this imaging satellite” and “all opportunities for this target object,” each within a time window — plus a one-item control record that says which run is currently live.

Data model: keys, index and control item



- Q0** Every request: GetItem RunState[CURRENT] to obtain g (cached ~60 s).
- Q1** Satellite X in window [t1, t2]: Query table, PK = SAT#X#G#g, SK BETWEEN TS#t1 AND TS#t2.
- Q2** Which satellites can image object Y: Query GSI, PK = OBJ#Y#G#g.

Figure 3. Data model: table, secondary index and control item.

7.1 Primary Table — ImagingOpportunities

Attribute	Example value	Role
PK	SAT#IMG-0042#G#2026092806	Partition key — imaging satellite + generation
SK	TS#2026-10-03T14:22:00Z#OBJ#DEB-19831	Sort key — opportunity time + object (sorts chronologically)
imaging_satellite_id	IMG-0042	Denormalized for convenience
target_object_id	DEB-19831	Denormalized for convenience
opportunity_datetime	2026-10-03T14:22:00Z	When the 100 km approach occurs
generation_id	2026092806	Which 6-hour run produced this item
ttl	epoch seconds	Auto-expiry (~24 hours) — see Section 7.3

7.2 Global Secondary Index — ByTargetObject

A GSI (all attributes projected) supports the reverse query — “which imaging satellites can capture this object, and when” — without scanning the main table:

```
PK = OBJ#{target_object_id}#G#{generation}
SK = TS#{opportunity_datetime}#SAT#{imaging_satellite_id}
```

Each write to the table also writes to the index, which is why the cost model counts two write units per item.

7.3 Freshness, Cutover and Retention

Each run writes into its own generation, which is part of the partition key. Nothing is visible to consumers until the run has finished and passed the quality gate (Section 14.2); only then does the state machine update the single RunState item (PK = CURRENT) to point at the new generation. The query Lambda reads that item (cached for about a minute) and queries only that generation.

- Consumers never see partial or mixed results: if a run fails or is aborted, the pointer simply stays on the previous generation.
- Rollback is a single-item update back to the previous generation.
- Writes are naturally idempotent: within a generation the keys are deterministic, so a retried batch overwrites the same items.
- Old generations expire through DynamoDB TTL (about 24 hours, so four generations are normally retained). TTL deletion consumes no write capacity and needs no cleanup job.

As Assumption A5 notes, the exercise places concurrency conflicts out of scope; the pointer is included as low-cost good practice, not as a conflict-resolution mechanism.

7.4 Query Sizing

A satellite holds about 13,500 items per generation (≈ 4.7 MB) and an object about 56. Queries are always bounded by a time window: the default (next 24 hours) returns about 1,930 items in a single page, and longer ranges are paginated, so response size and latency stay predictable. Object queries are tiny by comparison. The calculation is in Section 5.5.

8. End-to-End Workflow

Figure 4 shows one run as a sequence of interactions; the numbered list below gives the detail behind each step.

One 6-hour run: sequence of interactions

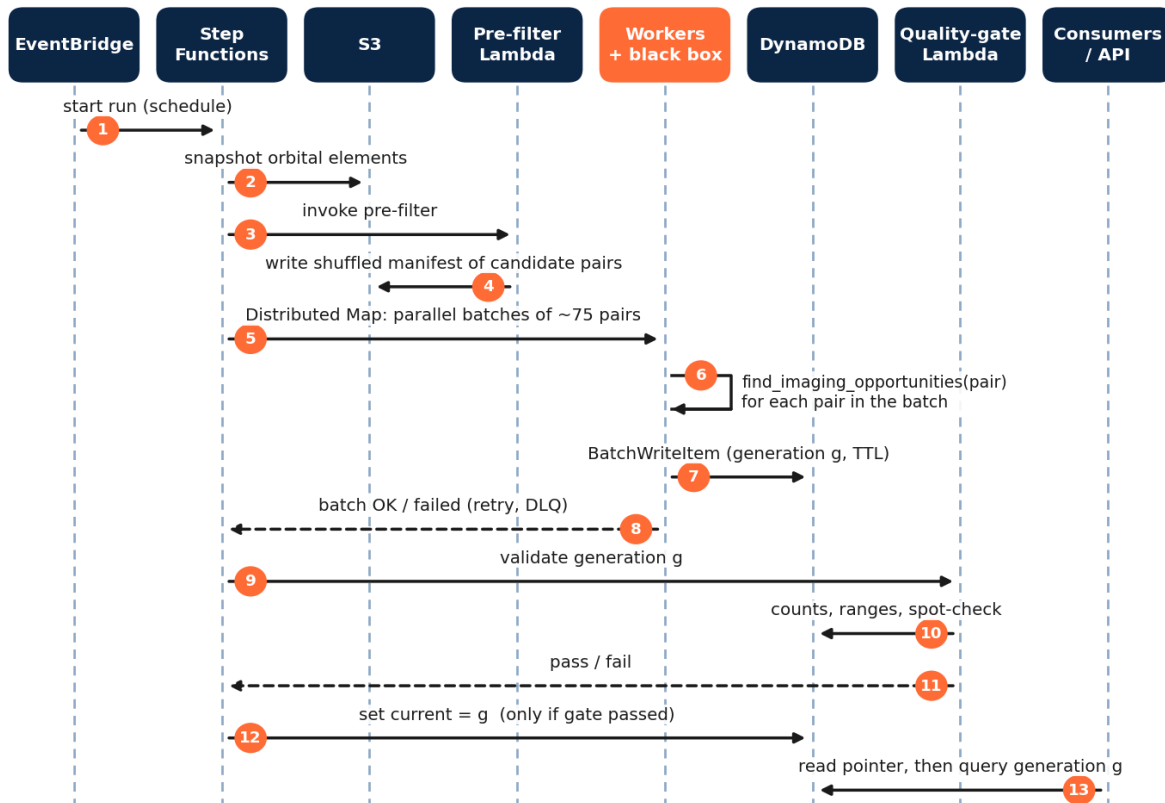


Figure 4. Sequence of interactions for one 6-hour run, from trigger to consumer query.

1. EventBridge Scheduler triggers the Step Functions orchestrator every 6 hours.
2. The orchestrator fetches current orbital elements for all imaging satellites and target objects, snapshots them to S3 and assigns the run's generation ID.
3. The pre-filter computes coarse orbital envelopes, prunes the full pair set down to geometrically plausible candidates and writes a shuffled manifest to S3.
4. Step Functions Distributed Map reads the manifest and fans out batches of candidate pairs to worker Lambdas.
5. Each worker calls `find_imaging_opportunities()` for every pair in its batch and writes the resulting datetimes to DynamoDB in the run's generation. Failed batches are retried; poison batches go to a dead-letter queue.
6. In parallel, results are written to S3 in Parquet for archival and ad-hoc analytics via Athena.
7. A quality-gate Lambda validates the completed generation (Section 14.2).
8. If the gate passes, the orchestrator updates the RunState pointer to the new generation; if not, an alarm is raised and the previous generation continues to be served.
9. Consumers query opportunities through API Gateway → Query Lambda → DynamoDB, by imaging satellite or by target object within a time window, with sub-second response times.
10. CloudWatch, Step Functions execution history and SNS alerts provide visibility into run health.

9. Scalability & Performance

9.1 Why This Scales

- Pre-filtering shrinks the expensive part of the workload before it reaches compute, so growth in the raw catalog (30,000 → 60,000 objects) does not translate one-to-one into more black-box calls.
- Step Functions Distributed Map and Lambda scale horizontally by design — going from 50 to 250 imaging satellites means more items in the manifest and more concurrent batches, not a re-architected pipeline.
- DynamoDB on-demand capacity scales its throughput automatically, and read latency stays flat because access is always by key within a bounded window, never by scan.

9.2 Meeting the 6-Hour Cadence

As calculated in Section 5.4, the compute stage needs only about 16 concurrent Lambda executions to fit a conservative 2-hour slice of the 6-hour budget at end-of-2026 scale, and Section 5.7 shows that even with a 1-second black box and no filter the requirement (about 2,084 workers) stays inside Distributed Map's 10,000 limit and needs only a Lambda quota increase. This leaves hours of headroom for retries, orbital-data delays or transient AWS throttling — deliberately, because a system that only just meets its SLA on a good day is not resilient.

9.3 Headroom Beyond 2026

Every stage (pre-filter, Distributed Map fan-out, DynamoDB on-demand) scales independently, so further growth in the satellite fleet or object catalog is absorbed by adding parallel workers and manifest items rather than by redesigning components.

10. Reliability & Fault Tolerance

- Step Functions Distributed Map provides per-batch retry with exponential backoff, so transient failures (throttling, timeouts) self-heal without custom logic, and a failure threshold aborts a run whose error rate is abnormal.
- Writes are idempotent: output is keyed by (satellite, generation, time, object), so re-processing a batch after a retry overwrites the same items rather than duplicating them.
- Results go live only when the whole generation has passed the automated quality gate and the pointer is flipped. If a run fails or cannot finish inside its window, the previous generation stays live — consumers always see a complete, consistent 7-day horizon that may be slightly stale, never partial.
- Persistent per-batch failures land in a dead-letter queue for inspection and re-drive rather than silently dropping opportunities.
- CloudWatch alarms on Step Functions failure rate, DynamoDB throttling, gate failures and the age of the current generation notify the on-call engineer through SNS.

11. Cost Considerations

The architecture is serverless end to end (EventBridge, Step Functions, Lambda, DynamoDB on-demand, S3), so cost is driven by the computation actually performed — four runs a day — rather than by a cluster provisioned around the clock. Section 5.6 estimates roughly \$1,000 per month at end-of-2026 scale under the

stated assumptions, split about evenly between Lambda compute (~\$481) and DynamoDB writes (~\$506), with storage, orchestration and the remaining services in the tens of dollars.

Cost levers, in order of impact: the pre-filter (about 6.7× on Lambda spend), Express rather than Standard child workflows (~\$360 per month avoided), Graviton (Arm) Lambda where the black box supports it (about 20% lower duration price), and provisioned capacity with auto scaling for DynamoDB once the write pattern is proven stable. If profiling shows the black box is far slower or heavier than assumed, AWS Batch with EC2 Spot becomes a strong alternative for the compute stage at very high sustained volume.

12. Monitoring & Observability

- Step Functions execution history gives a visual audit trail of every run, including which batches failed and why.
- CloudWatch Logs and custom metrics track pre-filter reduction ratio, black-box invocations, run duration, DynamoDB write throughput, and quality-gate results per run.
- AWS X-Ray traces requests through the query API to catch latency regressions before they threaten the sub-second SLA; a synthetic canary queries the API every five minutes.
- SNS notifies the team on run failure, run duration beyond a threshold (three hours, half the budget), gate failure, elevated DynamoDB throttling, or a current generation older than nine hours (one missed run plus margin).

13. Trade-offs & Alternatives Considered

The table summarizes the seven main decisions. Each row states what was chosen, what was rejected, why, and what the choice costs — because every option here gives something up.

Decision	Chosen	Alternatives	Why	What we give up
Compute engine	Lambda via Distributed Map (Express children)	AWS Batch / Fargate; EC2 fleet	Least operations; scales from zero to thousands in seconds; pay per use	15-minute and memory limits; harder to cost-optimize at very high sustained volume (Batch/Spot is the fallback)
Orchestration	Step Functions Distributed Map	SQS + worker fleet; Batch array jobs; EMR/Spark	Manifest-driven; built-in retries, failure thresholds and run history; no queue code	Less fine-grained back-pressure than a queue (mitigated by MaxConcurrency and a shuffled manifest)
Search space	Pre-filter, then parallel compute	Parallel brute force (no filter)	~6.7× lower compute cost; margin if calls are slower than assumed	An extra component to build; it must never drop a real opportunity (recall test)
Data store	DynamoDB, on-demand	Timestream; Aurora/RDS; OpenSearch	Key-based reads in single-digit ms; no capacity planning; TTL	No ad-hoc joins or aggregations (served by the S3 + Athena archive)
Write model	One item per opportunity	One item per pair holding a list of datetimes	Time-window queries, clean pagination, small items	Up to ~1.5× more writes than per-pair items; revisit if the function returns time-sampled datetimes
Cutover	Generation in key + validated pointer	Overwrite in place; one table per run	Atomic switch; never serves partial or mixed data; trivial rollback	~4× storage for retained generations (~9 GB, ~\$2 per month) and one cached read per request

Decision	Chosen	Alternatives	Why	What we give up
Batch size	75 pairs per invocation	1 pair; thousands per invocation	30,000 invocations instead of 2.25 million; a retry repeats only ~4 s of work	Larger batches slow retries; must shrink if per-call latency grows (A9)

13.1 Parallel Brute Force vs. Pre-Filtering

The first design considered was the simplest: call the black box for every raw pair. Section 5 shows that sequential execution is infeasible (8.68 days for one pass) but that parallel brute force is not: 15 million pairs at 50 ms need only about 105 concurrent workers for a 2-hour window, comfortably inside default Lambda limits. Parallelism, not the pre-filter, is therefore what makes the deadline achievable.

What the pre-filter buys is (a) about 6.7× lower compute cost at the baseline (roughly \$481 versus \$3,205 per month), (b) margin if the black box is slower than assumed — at 1 s per call the unfiltered job needs about 2,084 concurrent workers, above the default Lambda quota, while the baseline filter needs 313 — and (c) less load on every downstream service. Its costs are one more component to build and the absolute requirement that it never lose an opportunity. We recommend building it, but sequencing it after the parallel pipeline: the pipeline works with an identity filter, so the real filter can be introduced behind a flag and enabled only once the recall test passes on real data.

13.2 Lambda vs. AWS Batch / Fargate

Lambda is the default worker because the exercise treats the opportunity function as a simple, presumably stateless call; it offers the least operational overhead and the finest-grained automatic scaling. If the real function is long-running (approaching Lambda’s 15-minute limit) or needs more CPU or memory than Lambda offers, AWS Batch on Fargate or Spot EC2 is the natural substitute, and because the design is manifest-driven only the compute stage would change.

13.3 DynamoDB vs. Timestream and Relational Databases

Timestream was considered given the time-series flavour of opportunity datetimes, but the dominant query is a key lookup by satellite or object within a window, not time-range aggregation — a better fit for DynamoDB’s predictable point and range queries. A relational database such as Aurora adds connection management and scaling work that this access pattern does not need. Analytical questions are served from the S3 archive through Athena.

13.4 Write Model and Cutover

Storing one item per opportunity keeps items small and makes time-window queries and pagination trivial; a single item per pair would cut writes by at most about one third at the baseline but lose time-range queries and risks the 400 KB item limit if opportunity lists grow. Placing the generation in the key, with a validated pointer, costs a little storage and one cached read but removes an entire class of failure (serving partial or mixed data), which is why it is preferred to overwriting in place.

14. Automated Testing

The pipeline runs unattended four times a day and its output drives tasking decisions, so correctness must be verified continuously, not only at release. The strategy rests on three principles.

- Keep the black box behind a thin adapter (an OpportunityFinder interface) so every layer except contract and end-to-end tests can run against a fast, deterministic fake. Worker and API handlers take their dependencies (finder, table, clock) as parameters.
- Turn every property the design relies on into an executable check: the pre-filter never loses an opportunity, every candidate pair is processed, retries are harmless, the pointer flips only after validation, and reads meet the latency target.
- Run the checks in three places: in CI on every change, in staging on a schedule, and inside every production run as a gate.

14.1 Test Layers

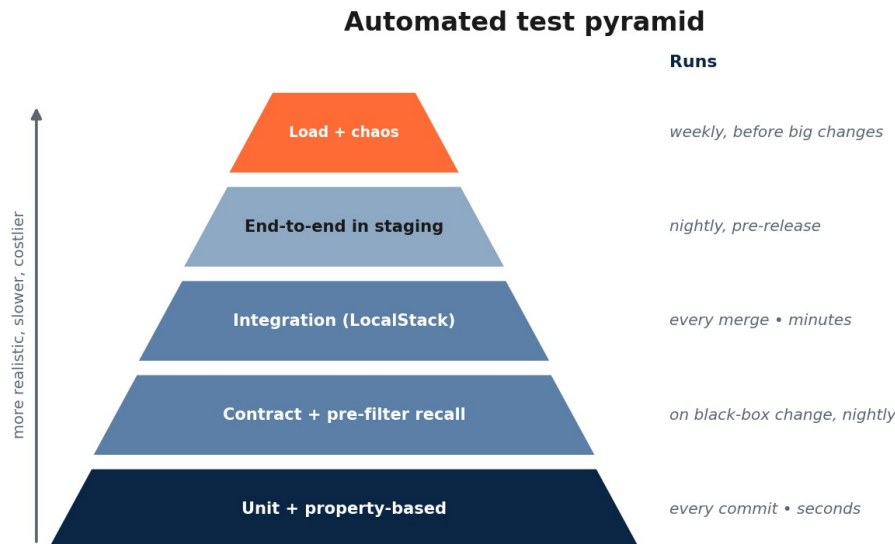


Figure 5. Automated test pyramid: many fast tests at the base, few realistic and costly tests at the top.

Layer	What it verifies	Tooling	When it runs
Unit	Pre-filter envelope maths and safety margin; batcher (sizes, no loss or duplication); key builders and chronological ordering of sort keys; TTL and generation helpers; worker handler with a fake finder; query handler validation and pagination	pytest, moto	Every commit
Property-based	Invariants over generated inputs: batches partition the manifest exactly; a larger margin never removes more pairs; keys round-trip; datetimes stay inside the horizon	Hypothesis	Every commit
Black-box contract	Adapter output is a sorted, unique list of timezone-aware UTC datetimes inside [t0, t0 + 7 days]; identical inputs give identical outputs (A6); an R&D-supplied golden set of ~50 pairs reproduces its expected results	pytest, golden files	On black-box version change; nightly
Pre-filter recall	On a stratified sample (~20,000 pairs, including near-threshold cases) every pair for which the real black box returns at least one opportunity survives the filter (recall = 100%); reports the measured reduction ratio, validating A2	pytest, real black box in parallel	Nightly; on filter or catalog change
Integration	Worker against DynamoDB Local / LocalStack: item counts, GSI symmetry (query by satellite and by object return the same set), idempotent retry, TTL and generation attributes. State machine with mocked tasks: retry, catch, failure threshold, DLQ routing	LocalStack, DynamoDB Local, Step Functions Local	Every merge

Layer	What it verifies	Tooling	When it runs
End-to-end (staging)	Full run on a scaled-down catalog (e.g. 5 satellites × 2,000 objects) with the real black box: execution succeeds; processed equals manifest count; API output equals an independent brute-force reference for the same snapshot; pointer flips only after the gate	Staging account, pytest, boto3	Nightly; before release
Load and performance	Full-scale synthetic catalog (250 × 60,000) with a stub finder of configurable latency (50 ms / 250 ms / 1 s) and opportunity density: run time, peak concurrency, DynamoDB throttling and hot partitions; API p95/p99 under load, including during ingest	k6 or Locust; seeded catalog generator	Weekly; before capacity-affecting changes
Resilience (chaos)	Injected faults: black-box exceptions (5% of calls), Lambda timeouts, DynamoDB throttling, S3 read errors, aborted execution. Retries recover, poison batches reach the DLQ and raise an alarm, a failed gate leaves the previous generation live	AWS Fault Injection Service where supported; fault-injecting fake finder	Weekly; before release
Infrastructure and security	Infrastructure-as-code assertions (keys, TTL, on-demand mode, alarms exist); policy scanning; least-privilege IAM checks	CDK assertions, cdk-nag / Checkov	Every commit

14.2 Runtime Quality Gate (Automated Testing in Production)

Before the pointer flips, a quality-gate Lambda invoked by the state machine validates the finished generation. If any check fails, the pointer does not move, an alarm is raised, and the previous generation keeps serving.

- Completeness: batches succeeded equals batches in the manifest, and the dead-letter queue is empty.
- Volume sanity: item count within $\pm 25\%$ of the previous validated generation (an initial threshold, tuned from history).
- Coverage: every imaging satellite has at least one opportunity; none indicates an ingestion problem rather than physics at this scale.
- Range and format: all datetimes fall inside $[t_0, t_0 + 7 \text{ days}]$; keys are well-formed; no duplicates.
- Spot check: 200 randomly chosen candidate pairs are re-run through the black box and compared with the stored items; an exact match is required, which catches silent write or read corruption.

A synthetic canary also queries the API every five minutes, asserting a non-empty result and latency under the target, and alarms on failure.

14.3 CI/CD Pipeline and Quality Gates

CI/CD pipeline with automated quality gates

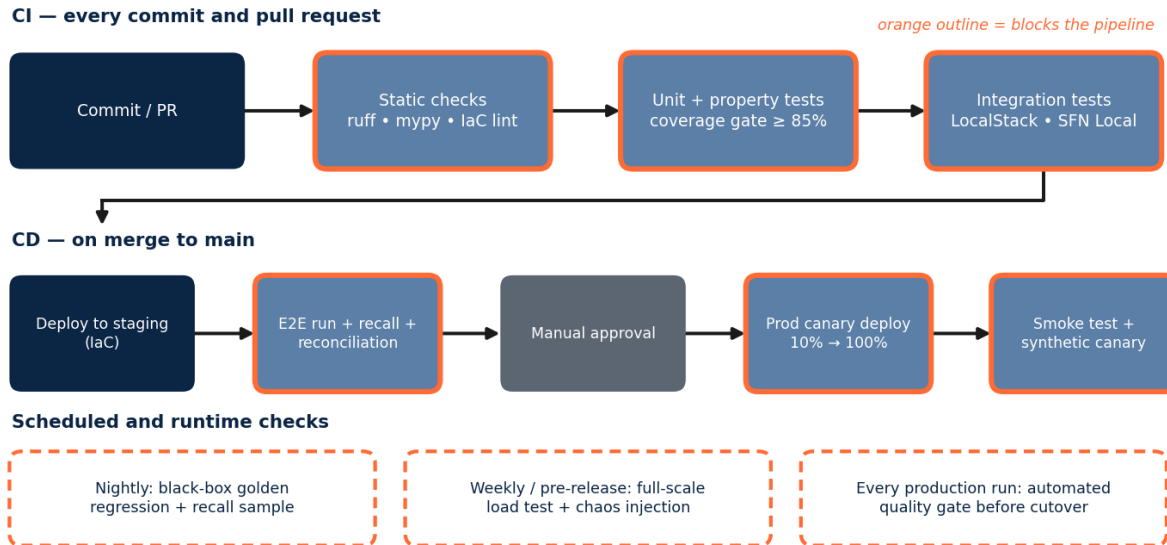


Figure 6. CI/CD pipeline. Stages outlined in orange block promotion when they fail.

Criterion	Target	Enforced by
Unit and property tests	100% pass; line coverage at least 85% overall and 100% branch coverage for the pre-filter, batcher and key builders	CI (blocking)
Pre-filter recall	100% on the sample; measured reduction ratio reported against the 85% planning value	Nightly and on change (blocking)
Idempotency and reconciliation	Re-running any batch leaves the table unchanged; end-to-end API results equal the brute-force reference	Integration and end-to-end (blocking)
Run duration at full scale	At most 60 minutes with a 250 ms stub finder (budget 6 h; production alarm at 3 h)	Weekly load test
DynamoDB throttling	Zero throttled requests during ingest	Load test and production alarm
Query latency	p95 under 300 ms and p99 under 1 s at expected request rate, including during ingest	Load test and synthetic canary
Failure handling	With 5% injected black-box failures a run either completes correctly or aborts safely with the pointer unchanged	Chaos test
Production gate	All checks in Section 14.2 pass before the pointer flips	Every production run

14.4 Example Tests

Two representative tests illustrate the style. The first guards the most important property of the pre-filter; the second guards the retry behaviour that the reliability design depends on.

```
def test_prefilter_never_drops_a_real_opportunity(snapshot, sample_pairs, finder):
    kept = set(prefilter(snapshot, sample_pairs, margin_km=100 + SAFETY_KM))
    missed = [p for p in sample_pairs if p not in kept and finder(*p)]
    assert not missed, f"{len(missed)} real opportunities dropped, e.g. {missed[:3]}"

def test_retried_batch_is_idempotent(ddb_table, fake_finder):
    event = {"generation": "2026092806",
            "pairs": [{"IMG-0042", "DEB-19831"}, {"IMG-0042", "DEB-20211"}]}
    handler(event, finder=fake_finder, table=ddb_table)
    first = scan_all(ddb_table)
    handler(event, finder=fake_finder, table=ddb_table)    # simulated retry
    assert scan_all(ddb_table) == first
```

14.5 Test Data and Environments

- Golden set: about 50 (satellite, object) pairs with expected outputs supplied and versioned by R&D, re-checked whenever the black box changes.
- Synthetic catalog generator: seeded, so any load or chaos run can be reproduced exactly; configurable size, altitude distribution, latency and opportunity density.
- Orbital snapshots and environments: fixed fixtures for unit and integration tests, a recent real snapshot for nightly recall and end-to-end tests, injected clocks, and an isolated staging account built from the same infrastructure code as production.

15. Future Enhancements

- Incremental recomputation: skip pairs involving a satellite or object whose orbital elements have not meaningfully changed since the last run, rather than recomputing the full catalog every cycle.
- Priority tiering: recompute high-value or highly maneuverable target objects more frequently than slow-changing debris, decoupling their refresh cadence from the blanket 6-hour cycle.
- Config-driven catalog growth: keep the satellite and object catalogs as external configuration and data, so scaling beyond 250 satellites or 60,000 objects requires no pipeline changes.
- Push notifications: stream newly discovered opportunities to a downstream tasking system via EventBridge or SNS as soon as each run's cutover completes, rather than requiring consumers to poll.

16. Summary

This design meets every stated requirement — AWS-hosted, black-box-compatible, a 6-hour recomputation cadence, and sub-second query latency — while staying honest about what is assumed. The decisions that matter most are: using serverless, natively parallel AWS primitives (Step Functions Distributed Map, Lambda, DynamoDB) so that a fivefold larger fleet and twofold larger catalog by the end of 2026 is a matter of configuration and throughput rather than a rebuild; reducing the search space with a conservative pre-filter that cuts compute cost about 6.7×; and publishing results only through an atomic, automatically validated cutover so that consumers never see partial data.

The five requested areas are covered in Sections 5.7, 6, 7, 8 and 14 (diagrams, Figures 1–6), 4 (assumptions), 5 (calculations), 13 (trade-offs) and 14 (automated testing). The first actions before building are to measure the black-box latency (A1) and the pre-filter reduction ratio (A2) on the real catalog.